

RELATORIO PI

CÓDIGO ESP32 :

```
#include <WiFi.h>
#include <WebServer.h>

WebServer server(80);

const char* ssid = "AnonDesk_UNIFEOB";
const char* password = "AcessoAnonDesk123";

// SUA INTERFACE EXATA (Convertida com segurança para o C++)
const String HTML_INTERFACE =
"<!DOCTYPE html>\n"
"<html lang=\"pt-BR\">\n"
"<head>\n"
"<meta charset=\"UTF-8\">\n"
"<meta name=\"viewport\" content=\"width=device-width, initial-scale=1.0\">\n"
"<title>AnonDesk · UNIFEOB</title>\n"
"<link rel=\"preconnect\" href=\"https://fonts.googleapis.com\">\n"
"<link"
href=\"https://fonts.googleapis.com/css2?family=DM+Serif+Display:ital@0;1&family=DM+Sans:wght@300;400;500;600&display=swap\" rel=\"stylesheet\">\n"
"<style>\n"
" :root {\n"
"   --blue-950: #0a1628;\n"
"   --blue-900: #0f2240;\n"
"   --blue-800: #163460;\n"
"   --blue-600: #1a56db;\n"
"   --blue-500: #2563eb;\n"
"   --blue-400: #3b82f6;\n"
"   --blue-300: #93c5fd;\n"
"   --blue-100: #dbeafe;\n"
"   --blue-50: #eff6ff;\n"
"   --white: #ffffff;\n"
"   --gray-200: #e2e8f0;\n"
"   --gray-400: #94a3b8;\n"
"   --gray-600: #475569;\n"
"   --success: #10b981;\n"
"   --error: #ef4444;\n"
"   --radius: 16px;\n"
"   --transition: 0.35s cubic-bezier(0.4,0,0.2,1);\n"
" }\n"
" *, ::before, ::after { box-sizing: border-box; margin: 0; padding: 0; }\n"
" html { scroll-behavior: smooth; }\n"
" body {\n"
```

```

" font-family: 'DM Sans', sans-serif;\n"
" background: var(--blue-950);\n"
" color: var(--white);\n"
" min-height: 100vh;\n"
" display: flex;\n"
" flex-direction: column;\n"
" align-items: center;\n"
" justify-content: center;\n"
" position: relative;\n"
" overflow: hidden;\n"
"}\n"
".bg-glow {\n"
" position: fixed; inset: 0; pointer-events: none; z-index: 0;\n"
"}\n"
".bg-glow::before {\n"
" content: ";\n"
" position: absolute;\n"
" top: -20%; left: 50%; transform: translateX(-50%);\n"
" width: 700px; height: 700px;\n"
" background: radial-gradient(circle, rgba(37,99,235,0.22) 0%, transparent 70%);\n"
" animation: pulse-glow 8s ease-in-out infinite alternate;\n"
"}\n"
".bg-glow::after {\n"
" content: ";\n"
" position: absolute;\n"
" bottom: -30%; right: -20%;\n"
" width: 500px; height: 500px;\n"
" background: radial-gradient(circle, rgba(59,130,246,0.12) 0%, transparent 70%);\n"
" animation: pulse-glow 11s ease-in-out infinite alternate-reverse;\n"
"}\n"
"@keyframes pulse-glow {\n"
" from { transform: translateX(-50%) scale(1); opacity: 0.7; }\n"
" to { transform: translateX(-50%) scale(1.15); opacity: 1; }\n"
"}\n"
".grid-overlay {\n"
" position: fixed; inset: 0; pointer-events: none; z-index: 0;\n"
" background-image:\n"
" linear-gradient(rgba(59,130,246,0.04) 1px, transparent 1px),\n"
" linear-gradient(90deg, rgba(59,130,246,0.04) 1px, transparent 1px);\n"
" background-size: 48px 48px;\n"
"}\n"
".page {\n"
" position: relative; z-index: 1;\n"
" width: 100%; max-width: 520px;\n"
" padding: 24px 20px 40px;\n"
" display: flex; flex-direction: column; align-items: center; gap: 32px;\n"
"}\n"
" header {\n"

```

```
" text-align: center;\n"
" display: flex; flex-direction: column; align-items: center; gap: 12px;\n"
" animation: slide-down 0.7s cubic-bezier(0.4,0,0.2,1) both;\n"
"}\n"
"@keyframes slide-down {\n"
" from { opacity: 0; transform: translateY(-24px); }\n"
" to { opacity: 1; transform: translateY(0); }\n"
"}\n"
".logo-wrap {\n"
" width: 56px; height: 56px;\n"
" background: linear-gradient(135deg, var(--blue-600), var(--blue-400));\n"
" border-radius: 18px;\n"
" display: flex; align-items: center; justify-content: center;\n"
" box-shadow: 0 0 0 6px rgba(59,130,246,0.15), 0 12px 32px rgba(37,99,235,0.35);\n"
" position: relative;\n"
"}\n"
".logo-wrap svg { width: 28px; height: 28px; }\n"
".badge {\n"
" display: inline-flex; align-items: center; gap: 6px;\n"
" font-size: 11px; font-weight: 600; letter-spacing: 0.08em;\n"
" text-transform: uppercase; color: var(--blue-300);\n"
" background: rgba(59,130,246,0.12);\n"
" border: 1px solid rgba(59,130,246,0.25);\n"
" border-radius: 100px; padding: 4px 12px;\n"
"}\n"
".badge::before {\n"
" content: ";\n"
" width: 6px; height: 6px;\n"
" background: var(--blue-400);\n"
" border-radius: 50%;\n"
" animation: blink 2s ease-in-out infinite;\n"
"}\n"
"@keyframes blink { 0%,100%{opacity:1} 50%{opacity:0.3} }\n"
"h1 {\n"
" font-family: 'DM Serif Display', serif;\n"
" font-size: clamp(2rem, 8vw, 2.6rem);\n"
" line-height: 1.1;\n"
" letter-spacing: -0.02em;\n"
" color: var(--white);\n"
"}\n"
"h1 em {\n"
" font-style: italic;\n"
" background: linear-gradient(135deg, var(--blue-300), var(--blue-400));\n"
" -webkit-background-clip: text; -webkit-text-fill-color: transparent;\n"
" background-clip: text;\n"
"}\n"
".subtitle {\n"
" font-size: 14px; color: var(--gray-400); line-height: 1.6;\n"
```

```
" max-width: 360px;\n"
"}\n"
".card {\n"
" width: 100%;\n"
" background: rgba(255,255,255,0.04);\n"
" border: 1px solid rgba(255,255,255,0.08);\n"
" border-radius: 24px;\n"
" padding: 28px 24px;\n"
" backdrop-filter: blur(20px);\n"
" -webkit-backdrop-filter: blur(20px);\n"
" box-shadow: 0 24px 64px rgba(0,0,0,0.35);\n"
" animation: slide-up 0.7s cubic-bezier(0.4,0,0.2,1) 0.15s both;\n"
"}\n"
"@keyframes slide-up {\n"
" from { opacity: 0; transform: translateY(28px); }\n"
" to { opacity: 1; transform: translateY(0); }\n"
"}\n"
".anon-bar {\n"
" display: flex; align-items: center; gap: 10px;\n"
" background: rgba(16,185,129,0.08);\n"
" border: 1px solid rgba(16,185,129,0.2);\n"
" border-radius: 12px;\n"
" padding: 10px 14px;\n"
" margin-bottom: 20px;\n"
"}\n"
".anon-bar svg { flex-shrink: 0; color: var(--success); }\n"
".anon-bar span { font-size: 12.5px; color: #6ee7b7; line-height: 1.4; }\n"
".field-label {\n"
" font-size: 12px; font-weight: 600;\n"
" text-transform: uppercase; letter-spacing: 0.07em;\n"
" color: var(--blue-300);\n"
" margin-bottom: 8px;\n"
" display: block;\n"
"}\n"
".textarea-wrap { position: relative; }\n"
"textarea {\n"
" width: 100%;\n"
" min-height: 140px;\n"
" background: rgba(15,34,64,0.7);\n"
" border: 1.5px solid rgba(59,130,246,0.2);\n"
" border-radius: 14px;\n"
" padding: 14px 16px 40px;\n"
" color: var(--white);\n"
" font-family: 'DM Sans', sans-serif;\n"
" font-size: 15px;\n"
" line-height: 1.65;\n"
" resize: vertical;\n"
" outline: none;\n"
}
```

```
" transition: border-color var(--transition), box-shadow var(--transition), background
var(--transition);\n"
" caret-color: var(--blue-400);\n"
" }\n"
" textarea::placeholder { color: rgba(148,163,184,0.5); }\n"
" textarea:focus {\n"
" border-color: var(--blue-500);\n"
" background: rgba(15,34,64,0.9);\n"
" box-shadow: 0 0 0 4px rgba(37,99,235,0.15);\n"
" }\n"
" textarea.error-state { border-color: var(--error); box-shadow: 0 0 0 4px
rgba(239,68,68,0.12); }\n"
" .char-count {\n"
" position: absolute; bottom: 10px; right: 14px;\n"
" font-size: 11px; color: var(--gray-400);\n"
" transition: color 0.2s;\n"
" pointer-events: none;\n"
" }\n"
" .char-count.warn { color: #fbbf24; }\n"
" .char-count.over { color: var(--error); }\n"
" .field-error {\n"
" font-size: 12px; color: #fca5a5;\n"
" margin-top: 6px; padding-left: 4px;\n"
" display: none;\n"
" }\n"
" .field-error.visible { display: block; }\n"
" .btn-submit {\n"
" width: 100%;\n"
" margin-top: 16px;\n"
" padding: 15px 24px;\n"
" background: linear-gradient(135deg, var(--blue-600) 0%, var(--blue-500) 100%);\n"
" border: none; border-radius: 14px;\n"
" color: var(--white);\n"
" font-family: 'DM Sans', sans-serif;\n"
" font-size: 15px; font-weight: 600;\n"
" cursor: pointer;\n"
" display: flex; align-items: center; justify-content: center; gap: 8px;\n"
" position: relative; overflow: hidden;\n"
" transition: transform 0.2s, box-shadow 0.2s, opacity 0.2s;\n"
" box-shadow: 0 4px 20px rgba(37,99,235,0.4);\n"
" }\n"
" .btn-submit:hover:not(:disabled) {\n"
" transform: translateY(-2px);\n"
" box-shadow: 0 8px 28px rgba(37,99,235,0.5);\n"
" }\n"
" .btn-submit:active:not(:disabled) { transform: translateY(0); }\n"
" .btn-submit:disabled { opacity: 0.6; cursor: not-allowed; }\n"
" .btn-submit .btn-shimmer {\n"
```

```

" position: absolute; inset: 0;\n"
" background: linear-gradient(105deg, transparent 40%, rgba(255,255,255,0.15) 50%,\n
transparent 60%);\n"
" transform: translateX(-100%);\n"
" transition: transform 0.6s;\n"
" }\n"
" .btn-submit:hover:not(:disabled) .btn-shimmer { transform: translateX(100%); }\n"
" .spinner {\n"
" width: 18px; height: 18px;\n"
" border: 2px solid rgba(255,255,255,0.3);\n"
" border-top-color: white;\n"
" border-radius: 50%;\n"
" animation: spin 0.7s linear infinite;\n"
" display: none;\n"
" }\n"
" @keyframes spin { to { transform: rotate(360deg); } }\n"
" .loading .spinner { display: block; }\n"
" .loading .btn-text { display: none; }\n"
" .toast {\n"
" position: fixed;\n"
" top: 24px; left: 50%; transform: translateX(-50%) translateY(-100px);\n"
" z-index: 100;\n"
" min-width: 280px; max-width: 90vw;\n"
" padding: 14px 20px;\n"
" border-radius: 14px;\n"
" display: flex; align-items: flex-start; gap: 12px;\n"
" font-size: 14px; font-weight: 500;\n"
" box-shadow: 0 16px 48px rgba(0,0,0,0.4);\n"
" transition: transform 0.5s cubic-bezier(0.34,1.56,0.64,1), opacity 0.5s;\n"
" opacity: 0;\n"
" pointer-events: none;\n"
" }\n"
" .toast.show {\n"
" transform: translateX(-50%) translateY(0);\n"
" opacity: 1;\n"
" pointer-events: auto;\n"
" }\n"
" .toast-success {\n"
" background: #064e3b;\n"
" border: 1px solid rgba(16,185,129,0.35);\n"
" color: #a7f3d0;\n"
" }\n"
" .toast-error {\n"
" background: #450a0a;\n"
" border: 1px solid rgba(239,68,68,0.35);\n"
" color: #fca5a5;\n"
" }\n"
" .toast-icon { flex-shrink: 0; margin-top: 1px; }\n"

```

```

" .toast-msg { line-height: 1.5; }\n"
" .success-overlay {\n"
"   display: none;\n"
"   flex-direction: column; align-items: center; justify-content: center;\n"
"   text-align: center; gap: 16px;\n"
"   padding: 12px 0;\n"
"   animation: fade-in 0.5s ease both;\n"
" }\n"
" @keyframes fade-in { from{opacity:0;transform:scale(0.96)}\nto{opacity:1;transform:scale(1)} }\n"
" .success-overlay.active { display: flex; }\n"
" .card-form.hidden { display: none; }\n"
" .checkmark-circle {\n"
"   width: 72px; height: 72px;\n"
"   background: rgba(16,185,129,0.1);\n"
"   border: 2px solid rgba(16,185,129,0.3);\n"
"   border-radius: 50%;\n"
"   display: flex; align-items: center; justify-content: center;\n"
"   animation: pop 0.5s cubic-bezier(0.34,1.56,0.64,1) both;\n"
" }\n"
" @keyframes pop { from{transform:scale(0)} to{transform:scale(1)} }\n"
" .checkmark-circle svg { color: var(--success); }\n"
" .success-title {\n"
"   font-family: 'DM Serif Display', serif;\n"
"   font-size: 1.5rem;\n"
"   color: var(--white);\n"
" }\n"
" .success-sub { font-size: 13.5px; color: var(--gray-400); line-height: 1.5; max-width: 280px; }\n"
" .btn-new {\n"
"   margin-top: 8px;\n"
"   padding: 12px 28px;\n"
"   background: transparent;\n"
"   border: 1.5px solid rgba(59,130,246,0.4);\n"
"   border-radius: 12px;\n"
"   color: var(--blue-300);\n"
"   font-family: 'DM Sans', sans-serif;\n"
"   font-size: 14px; font-weight: 500;\n"
"   cursor: pointer;\n"
"   transition: all var(--transition);\n"
" }\n"
" .btn-new:hover {\n"
"   background: rgba(59,130,246,0.1);\n"
"   border-color: var(--blue-400);\n"
"   color: var(--white);\n"
" }\n"
" .queue-info {\n"
"   display: flex; align-items: center; justify-content: center; gap: 8px;\n"

```

```

" font-size: 12px; color: var(--gray-400);\n"
" animation: slide-up 0.7s cubic-bezier(0.4,0,0.2,1) 0.3s both;\n"
" }\n"
" .queue-dot { width:8px; height:8px; border-radius:50%; background:var(--blue-400);
animation: blink 2s infinite; }\n"
" footer {\n"
" font-size: 11px; color: rgba(148,163,184,0.4);\n"
" text-align: center; letter-spacing: 0.04em;\n"
" animation: slide-up 0.7s cubic-bezier(0.4,0,0.2,1) 0.4s both;\n"
" }\n"
" @media (max-width: 400px) {\n"
" .card { padding: 22px 18px; }\n"
" h1 { font-size: 1.75rem; }\n"
" }\n"
"</style>\n"
"</head>\n"
"<body>\n"
"<div class=\"bg-glow\"></div>\n"
"<div class=\"grid-overlay\"></div>\n"
"<div class=\"toast toast-success\" id=\"toast\" role=\"alert\" aria-live=\"polite\">\n"
" <span class=\"toast-icon\" id=\"toast-icon\"></span>\n"
" <span class=\"toast-msg\" id=\"toast-msg\"></span>\n"
"</div>\n"
"<main class=\"page\">\n"
" <header>\n"
" <div class=\"logo-wrap\" aria-hidden=\"true\">\n"
" <svg viewBox=\"0 0 24 24\" fill=\"none\" stroke=\"white\" stroke-width=\"2\"
stroke-linecap=\"round\" stroke-linejoin=\"round\">\n"
" <path d=\"M21 15a2 2 0 0 1-2 2H7l-4 4V5a2 2 0 0 1 2-2h14a2 2 0 0 1 2 2z\"/>\n"
" <line x1=\"12\" y1=\"8\" x2=\"12\" y2=\"12\"/><line x1=\"12\" y1=\"16\"
x2=\"12.01\" y2=\"16\"/>\n"
" </svg>\n"
" </div>\n"
" <div class=\"badge\">Perguntas ao Vivo</div>\n"
" <h1>Anon<em>Desk</em></h1>\n"
" <p class=\"subtitle\">Envie sua dúvida anonimamente. Ela aparecerá em tempo
real no display do professor.</p>\n"
" </header>\n"
" <div class=\"card\" role=\"main\">\n"
" <div class=\"card-form\" id=\"card-form\">\n"
" <div class=\"anon-bar\" aria-label=\"Garantia de anonimato\">\n"
" <svg width=\"16\" height=\"16\" viewBox=\"0 0 24 24\" fill=\"none\"
stroke=\"currentColor\" stroke-width=\"2.5\" stroke-linecap=\"round\" stroke-
linejoin=\"round\">\n"
" <rect x=\"3\" y=\"11\" width=\"18\" height=\"11\" rx=\"2\" ry=\"2\"/>\n"
" <path d=\"M7 11V7a5 5 0 0 1 10 0v4\"/>\n"
" </svg>\n"

```

```

"    <span>Sua identidade não é coletada nem armazenada. Nenhum dado pessoal é
enviado.</span>\n"
"  </div>\n"
"    <label class=\"field-label\" for=\"question\">Sua pergunta</label>\n"
"    <div class=\"textarea-wrap\">\n"
"      <textarea\n"
"        id=\"question\"\n"
"        name=\"question\"\n"
"        placeholder=\"Digite sua dúvida aqui... Ex: Poderia explicar novamente o
conceito de herança em orientação a objetos?\"\n"
"        maxlength=\"500\"\n"
"        aria-required=\"true\"\n"
"        aria-describedby=\"field-error char-count\"\n"
"      ></textarea>\n"
"      <span class=\"char-count\" id=\"char-count\" aria-live=\"polite\">0 /
500</span>\n"
"    </div>\n"
"    <span class=\"field-error\" id=\"field-error\" role=\"alert\">Por favor, escreva sua
pergunta antes de enviar.</span>\n"
"    <button class=\"btn-submit\" id=\"btn-submit\" type=\"button\" aria-label=\"Enviar
pergunta\">\n"
"      <div class=\"btn-shimmer\"></div>\n"
"      <span class=\"btn-text\" id=\"btn-text\">\n"
"        <svg width=\"16\" height=\"16\" viewBox=\"0 0 24 24\" fill=\"none\"
stroke=\"currentColor\" stroke-width=\"2.5\" stroke-linecap=\"round\" stroke-
linejoin=\"round\" style=\"margin-right:4px;vertical-align:-2px\">\n"
"          <line x1=\"22\" y1=\"2\" x2=\"11\" y2=\"13\"/><polygon points=\"22 2 15 22 11
13 2 9 22 2\"/>\n"
"        </svg>\n"
"        Enviar Pergunta\n"
"      </span>\n"
"      <div class=\"spinner\" id=\"spinner\" aria-hidden=\"true\"></div>\n"
"    </button>\n"
"  </div>\n"
"  <div class=\"success-overlay\" id=\"success-overlay\" aria-live=\"assertive\">\n"
"    <div class=\"checkmark-circle\">\n"
"      <svg width=\"36\" height=\"36\" viewBox=\"0 0 24 24\" fill=\"none\"
stroke=\"currentColor\" stroke-width=\"2.5\" stroke-linecap=\"round\" stroke-
linejoin=\"round\">\n"
"        <polyline points=\"20 6 9 17 4 12\"/>\n"
"      </svg>\n"
"    </div>\n"
"    <p class=\"success-title\">Pergunta enviada!</p>\n"
"    <p class=\"success-sub\">Sua dúvida foi recebida e será exibida no display do
professor em instantes.</p>\n"
"    <button class=\"btn-new\" id=\"btn-new\" type=\"button\">Enviar outra
pergunta</button>\n"
"  </div>\n"

```

```

" </div>\n"
" <div class=\"queue-info\" id=\"queue-info\" aria-live=\"polite\">\n"
"   <div class=\"queue-dot\"></div>\n"
"   <span id=\"queue-text\">Conectando ao servidor...</span>\n"
" </div>\n"
" <footer>UNIFEOB · AnonDesk &copy; 2025 · Open Source</footer>\n"
"</main>\n"
"<script>\n"
"  const API_URL   = '/api/perguntas';\n"
"  const WS_URL    = window.location.origin;\n"
"  const MAX_CHARS = 500;\n"
"  const WARN_CHARS = 400;\n"
"  const questionEl = document.getElementById('question');\n"
"  const charCountEl = document.getElementById('char-count');\n"
"  const fieldErrorEl = document.getElementById('field-error');\n"
"  const btnSubmit   = document.getElementById('btn-submit');\n"
"  const btnText     = document.getElementById('btn-text');\n"
"  const spinner     = document.getElementById('spinner');\n"
"  const cardForm    = document.getElementById('card-form');\n"
"  const successOver = document.getElementById('success-overlay');\n"
"  const btnNew      = document.getElementById('btn-new');\n"
"  const queueText   = document.getElementById('queue-text');\n"
"  const toastEl     = document.getElementById('toast');\n"
"  const toastMsgEl  = document.getElementById('toast-msg');\n"
"  const toastIconEl = document.getElementById('toast-icon');\n"
"  let toastTimer = null;\n"
"  let queueCount = 0;\n"
"  questionEl.addEventListener('input', () => {\n"
"    const len = questionEl.value.length;\n"
"    charCountEl.textContent = `${len} / ${MAX_CHARS}`;\n"
"    charCountEl.className = 'char-count' + (len >= MAX_CHARS ? ' over' : len >=
WARN_CHARS ? ' warn' : '');\n"
"    if (len > 0) {\n"
"      questionEl.classList.remove('error-state');\n"
"      fieldErrorEl.classList.remove('visible');\n"
"    }\n"
"  });\n"
"  function initSocket() {\n"
"    if (typeof io === 'undefined') {\n"
"      queueText.textContent = 'Sistema pronto para receber perguntas.';\n"
"      return;\n"
"    }\n"
"    const socket = io(WS_URL, { transports: ['websocket'], reconnectionDelay: 2000
});\n"
"    socket.on('connect', () => { queueText.textContent = 'Conectado · Sistema ao vivo';
});\n"
"    socket.on('disconnect', () => { queueText.textContent = 'Reconectando...'; });\n"
"    socket.on('queue_update', (data) => {\n"

```

```

"   queueCount = data.count ?? 0;\n"
"   queueText.textContent = queueCount === 0\n"
"     ? 'Nenhuma pergunta na fila agora.\n"
"       : `${queueCount} pergunta${queueCount > 1 ? 's' : ''} na fila`;\n"
"   });\n"
"   socket.on('connect_error', () => { queueText.textContent = 'Sistema disponível · Sem tempo real'; });\n"
" }\n"
" initSocket();\n"
" btnSubmit.addEventListener('click', handleSubmit);\n"
" questionEl.addEventListener('keydown', (e) => {\n"
"   if (e.key === 'Enter' && (e.ctrlKey || e.metaKey)) handleSubmit();\n"
" });\n"
" async function handleSubmit() {\n"
"   const text = questionEl.value.trim();\n"
"   if (!text) {\n"
"     questionEl.classList.add('error-state');\n"
"     fieldErrorEl.classList.add('visible');\n"
"     questionEl.focus();\n"
"     return;\n"
"   }\n"
"   if (text.length > MAX_CHARS) {\n"
"     showToast('error', 'A pergunta excede o limite de 500 caracteres.);\n"
"     return;\n"
"   }\n"
"   setLoading(true);\n"
"   try {\n"
"     const response = await fetch(API_URL, {\n"
"       method: 'POST',\n"
"       headers: { 'Content-Type': 'application/json' },\n"
"       body: JSON.stringify({ text })\n"
"     });\n"
"     if (!response.ok) throw new Error(`Servidor retornou ${response.status}`);\n"
"     cardForm.classList.add('hidden');\n"
"     successOver.classList.add('active');\n"
"   } catch (err) {\n"
"     console.warn('Erro ao enviar:', err);\n"
"     if (err.message.includes('Failed to fetch') || err.message.includes('NetworkError'))\n"
"   {\n"
"     cardForm.classList.add('hidden');\n"
"     successOver.classList.add('active');\n"
"   } else {\n"
"     showToast('error', 'Não foi possível enviar. Verifique sua conexão e tente novamente.);\n"
"   }\n"
"   } finally {\n"
"     setLoading(false);\n"
"   }\n"

```

```

" }\n"
" btnNew.addEventListener('click', () => {\n"
"   questionEl.value = ""; \n"
"   charCountEl.textContent = '0 / 500'; \n"
"   charCountEl.className = 'char-count'; \n"
"   fieldErrorEl.classList.remove('visible'); \n"
"   questionEl.classList.remove('error-state'); \n"
"   successOver.classList.remove('active'); \n"
"   cardForm.classList.remove('hidden'); \n"
"   questionEl.focus(); \n"
" }); \n"
" function setLoading(state) {\n"
"   btnSubmit.disabled = state; \n"
"   btnSubmit.classList.toggle('loading', state); \n"
"   spinner.style.display = state ? 'block' : 'none'; \n"
"   btnText.style.display = state ? 'none' : 'flex'; \n"
" } \n"
" function showToast(type, msg) {\n"
"   clearTimeout(toastTimer); \n"
"   toastEl.className = 'toast toast-' + type; \n"
"   toastMsgEl.textContent = msg; \n"
"   toastIconEl.innerHTML = type === 'success' \n"
"     ? `<svg width="18" height="18" viewBox="0 0 24 24" fill="none" \n"
stroke="#34d399" stroke-width="2.5" stroke-linecap="round" stroke- \n"
linejoin="round"><polyline points="20 6 9 17 4 12"/></svg>` \n"
"     : `<svg width="18" height="18" viewBox="0 0 24 24" fill="none" \n"
stroke="#f87171" stroke-width="2.5" stroke-linecap="round" stroke- \n"
linejoin="round"><circle cx="12" cy="12" r="10"/><line x1="15" y1="9" \n"
x2="9" y2="15"/><line x1="9" y1="9" x2="15" y2="15"/></svg>`; \n"
"   toastEl.classList.add('show'); \n"
"   toastTimer = setTimeout(() => toastEl.classList.remove('show'), 4500); \n"
" } \n"
"</script>\n"
"</body>\n"
"</html>";

```

// Função para ler o formato JSON enviado exatamente pela sua página

```

String extrairTextoDoJson(String json) {
  int posChave = json.indexOf("\"text\"");
  if (posChave == -1) return "";
  int posDoisPontos = json.indexOf(":", posChave);
  int posAspasInicio = json.indexOf("\"", posDoisPontos);
  if (posAspasInicio == -1) return "";
  int posAspasFim = json.indexOf("\"", posAspasInicio + 1);
  if (posAspasFim == -1) return "";
  return json.substring(posAspasInicio + 1, posAspasFim);
}

```

```

void setup() {
  Serial2.begin(9600, SERIAL_8N1, 16, 17);
  WiFi.softAP(ssid, password);

  server.on("/", []() {
    server.send(200, "text/html", HTML_INTERFACE);
  });

  // Endpoint que processa o envio exatamente como o seu JS espera
  server.on("/api/perguntas", HTTP_POST, []() {
    if (server.hasArg("plain")) {
      String corpoRequisicao = server.arg("plain");
      String msgDoAluno = extrairTextoDoJson(corpoRequisicao);

      if (msgDoAluno != "") {
        msgDoAluno.replace("\r", " ");
        msgDoAluno.replace("\n", " ");
        msgDoAluno.trim();

        Serial2.println(msgDoAluno);
        server.send(200, "application/json", "{\"status\":\"ok\"}");
      } else {
        server.send(400, "application/json", "{\"error\":\"Texto nao encontrado\"}");
      }
    } else {
      server.send(400, "application/json", "{\"error\":\"Corpo vazio\"}");
    }
  });

  server.begin();
}

void loop() {
  server.handleClient();
}

```

CÓDIGO ARDUINO MEGA:

```
#include <MCUFRIEND_kbv.h>
#include <Adafruit_GFX.h>
#include <SoftwareSerial.h>

MCUFRIEND_kbv tft;
SoftwareSerial espSerial(51, 53); // Pinos corretos que estamos usando

#define COLOR_BG      0x0842
#define COLOR_CARD     0x0924
#define COLOR_HEADER   0x00E2
#define COLOR_BORDER   0x11A8
#define COLOR_TEXT_MAIN 0xFFFF
#define COLOR_TEXT_MUTED 0x94B2
#define COLOR_BRAND    0x3BFF
#define COLOR_SUCCESS   0x15E8

String listaPerguntas[50];
int totalRecebido = 0;
int indiceAtual = -1;

const int pinoBotao = 49;
unsigned long tempoUltimoClique = 0;

void setup() {
  pinMode(pinoBotao, INPUT_PULLUP);
  espSerial.begin(9600);

  uint16_t idDisplay = tft.readID();
  if(idDisplay == 0x0101) idDisplay = 0x9486;
  tft.begin(idDisplay);
  tft.setRotation(1);
  desenharPainelBoasVindas();
}

void loop() {
  // Escuta as mensagens do Wi-Fi
  if (espSerial.available() > 0) {
    String novaPergunta = espSerial.readStringUntil('\n');
    novaPergunta.trim();

    if (novaPergunta.length() > 2 && totalRecebido < 50) {
      listaPerguntas[totalRecebido] = novaPergunta;
      if (totalRecebido == 0) {
        indiceAtual = 0;
      }
    }
  }
}
```

```

    totalRecebido++;
    renderizarPainelProfessor();
}
}

// — LÓGICA DO BOTÃO (SUPER SIMPLIFICADA E À PROVA DE FALHAS) —
bool estadoBotao = digitalRead(pinoBotao);
unsigned long tempoAtual = millis();

// Se o botão for pressionado (LOW) e passou o tempo de segurança anti-ruído
(300ms)
if (estadoBotao == LOW && (tempoAtual - tempoUltimoClique > 300)) {
    tempoUltimoClique = tempoAtual; // Grava o momento do clique

    // Se existir pelo menos 1 pergunta na memória, ele avança
    if (totalRecebido > 0) {
        indiceAtual++;

        // Se avançou além da última pergunta, ele faz o giro e volta para a primeira
        (Carrossel Infinito)
        if (indiceAtual >= totalRecebido) {
            indiceAtual = 0;
        }
        renderizarPainelProfessor();
    }
}

// — COMPONENTES DE DESIGN (MANTIDOS INTACTOS) —

void desenharEstruturaBase() {
    tft.fillScreen(COLOR_BG);
    tft.fillRect(0, 0, 480, 55, COLOR_HEADER);
    tft.drawFastHLine(0, 55, 480, COLOR_BORDER);
    tft.fillRect(20, 16, 22, 16, COLOR_BRAND);
    tft.fillTriangle(24, 32, 30, 32, 24, 37, COLOR_BRAND);
    tft.setTextColor(COLOR_TEXT_MAIN); tft.setTextSize(2);
    tft.setCursor(55, 17); tft.print("AnonDesk");
    tft.setTextColor(COLOR_TEXT_MUTED); tft.setTextSize(1);
    tft.setCursor(160, 22); tft.print("| MURAL DE PERGUNTAS");
}

void desenharPainelBoasVindas() {
    desenharEstruturaBase();
    int cX = 35, cY = 95, cW = 410, cH = 130;
    tft.drawRoundRect(cX, cY, cW, cH, 12, COLOR_BORDER);
    tft.fillRect(cX + 1, cY + 1, cW - 2, cH - 2, COLOR_CARD);
    tft.fillCircle(cX + 35, cY + 65, 6, COLOR_SUCCESS);
}

```

```

tft.drawCircle(cX + 35, cY + 65, 11, COLOR_SUCCESS);
tft.setTextColor(COLOR_TEXT_MAIN); tft.setTextSize(2);
tft.setCursor(cX + 65, cY + 45); tft.print("Aguardando Perguntas");
tft.setTextColor(COLOR_TEXT_MUTED); tft.setTextSize(1);
tft.setCursor(cX + 65, cY + 74); tft.print("O painel esta pronto.");

tft.drawFastHLine(0, 260, 480, COLOR_BORDER);
tft.fillRect(0, 261, 480, 59, COLOR_HEADER);
tft.setTextSize(1); tft.setCursor(15, 275);
tft.setTextColor(COLOR_TEXT_MUTED); tft.print("REDE: ");
tft.setTextColor(COLOR_BRAND); tft.print("AnonDesk_UNIFEOB");
tft.setCursor(250, 275);
tft.setTextColor(COLOR_TEXT_MUTED); tft.print("SENHA: ");
tft.setTextColor(COLOR_BRAND); tft.print("AcessoAnonDesk123");
tft.setCursor(120, 298);
tft.setTextColor(COLOR_TEXT_MUTED); tft.print("IP DE ACESSO: ");
tft.setTextColor(COLOR_SUCCESS); tft.print("http://192.168.4.1");
}

void renderizarPainelProfessor() {
    desenharEstruturaBase();
    if (totalRecebido == 0) return;

    int cardX = 20, cardY = 80, cardW = 440, cardH = 165;
    tft.drawRoundRect(cardX, cardY, cardW, cardH, 14, COLOR_BORDER);
    tft.fillRect(cardX + 1, cardY + 1, cardW - 2, cardH - 2, COLOR_CARD);
    tft.setTextColor(COLOR_TEXT_MAIN); tft.setTextSize(2);
    int paddingX = cardX + 22, paddingY = cardY + 22;
    tft.setCursor(paddingX, paddingY);
    String textoExibicao = listaPerguntas[indiceAtual];

    for (int i = 0; i < textoExibicao.length(); i++) {
        if (tft.getCursorX() > (cardX + cardW - 25)) {
            paddingY += 24; tft.setCursor(paddingX, paddingY);
            if (paddingY > (cardY + cardH - 30)) { tft.print("..."); break; }
        }
        tft.print(textoExibicao[i]);
    }

    tft.drawFastHLine(0, 270, 480, COLOR_BORDER);
    tft.fillRect(0, 271, 480, 49, COLOR_HEADER);
    tft.setTextColor(COLOR_TEXT_MUTED); tft.setTextSize(1);
    tft.setCursor(25, 288); tft.print("PERGUNTA ");
    tft.setTextColor(COLOR_BRAND); tft.setTextSize(2);
    tft.setCursor(95, 283); tft.print(String(indiceAtual + 1));
    tft.setTextColor(COLOR_TEXT_MUTED); tft.setTextSize(1);
    tft.setCursor(125, 288); tft.print("DE");
    tft.setTextColor(COLOR_TEXT_MAIN); tft.setTextSize(2);

```

```
tft.setCursor(145, 283); tft.print(String(totalRecebido));

int badgeX = 345, badgeY = 281, badgeW = 110, badgeH = 26;
tft.drawRoundRect(badgeX, badgeY, badgeW, badgeH, 6, COLOR_BORDER);
tft.fillRect(badgeX + 1, badgeY + 1, badgeW - 2, badgeH - 2, COLOR_BG);
tft.fillCircle(badgeX + 15, badgeY + 13, 4, COLOR_SUCCESS);
tft.setTextColor(COLOR_TEXT_MUTED); tft.setTextSize(1);
tft.setCursor(badgeX + 28, badgeY + 9); tft.print("LIVE FEED");
}
```